

Agent Phantom Recovery

13-api-specification.md

Production API Specification

Version: 1.0

Part 2

13. Task APIs

Tasks are the primary execution resource in the platform.

Everything ultimately maps back to a task.

Create Task

Endpoint

```
POST /api/v1/tasks
```

Request

```
{
  "project_id": "uuid",
  "goal": "Investigate authentication failures and generate a fix",
  "priority": 5
}
```

Response

```
{
  "task_id": "uuid",
  "status": "CREATED"
}
```

Get Task

Endpoint

```
GET /api/v1/tasks/{taskId}
```

Response

```
{
  "task_id": "",
  "goal": "",
  "status": "ANALYZING",
  "priority": 5,
  "created_at": ""
}
```

List Tasks

Endpoint

```
GET /api/v1/tasks
```

Query Parameters

```
?project_id=
?status=
?page=
?page_size=
```

Pause Task

Endpoint

```
POST /api/v1/tasks/{taskId}/pause
```

Response

```
{
  "status": "PAUSED"
}
```

Resume Task

Endpoint

```
POST /api/v1/tasks/{taskId}/resume
```

Cancel Task

Endpoint

```
POST /api/v1/tasks/{taskId}/cancel
```

Retry Task

Endpoint

```
POST /api/v1/tasks/{taskId}/retry
```

Task Status Lifecycle

```
CREATED
↓
PLANNING
↓
COLLECTING_EVIDENCE
↓
ANALYZING
↓
EXECUTING
↓
VERIFYING
↓
COMPLETED
```

Alternative paths:

FAILED
BLOCKED
PAUSED

14. Task Timeline APIs

Provides the Antigravity timeline workspace.

Get Task Timeline

Endpoint

```
GET /api/v1/tasks/{taskId}/timeline
```

Response

```
{
  "events": [
    {
      "timestamp": "",
      "event": "TaskCreated"
    }
  ]
}
```

Get Task Checkpoints

Endpoint

```
GET /api/v1/tasks/{taskId}/checkpoints
```

Restore Checkpoint

Endpoint

```
POST /api/v1/tasks/{taskId}/restore
```

Request

```
{
  "checkpoint_id": ""
}
```

15. Plan APIs

Plans are generated by the Planner service.

Generate Plan

Endpoint

```
POST /api/v1/tasks/{taskId}/plan
```

Response

```
{
  "plan_id": "",
  "version": 1,
  "status": "GENERATED"
}
```

Get Current Plan

Endpoint

```
GET /api/v1/tasks/{taskId}/plan
```

Response

```
{
  "plan_id": "",
  "steps": []
}
```

Get Plan Versions

Endpoint

```
GET /api/v1/tasks/{taskId}/plans
```

Replan Task

Endpoint

```
POST /api/v1/tasks/{taskId}/replan
```

Purpose

Used when:

- evidence changes
 - assumptions fail
 - execution becomes blocked
-

Plan Schema

```
{
  "plan_id": "",
  "steps": [
    {
      "step_id": "",
      "order": 1,
      "description": "",
      "tool": "",
      "status": ""
    }
  ]
}
```

16. Memory APIs

Memory is treated as a first-class system.

Retrieve Context

Endpoint

```
POST /api/v1/memory/retrieve
```

Request

```
{
  "project_id": "",
  "query": "authentication middleware"
}
```

Response

```
{
  "memories": []
}
```

Store Memory

Endpoint

```
POST /api/v1/memory/store
```

Request

```
{
  "project_id": "",
  "memory_type": "PROJECT",
  "content": ""
}
```

Update Memory

Endpoint

```
PATCH /api/v1/memory/{memoryId}
```

Delete Memory

Endpoint

```
DELETE /api/v1/memory/{memoryId}
```

Memory Search

Endpoint

```
POST /api/v1/memory/search
```

Request

```
{  
  "query": ""  
}
```

Memory Types

```
WORKING  
SESSION  
PROJECT  
EXPERIENCE
```

17. Agent APIs

Provider-independent agent access.

Agent Status

Endpoint

```
GET /api/v1/agents
```


Response

```
{
  "planner": {},
  "reasoner": {},
  "verifier": {}
}
```

Agent Health

Endpoint

```
GET /api/v1/agents/health
```

Agent Metrics

Endpoint

```
GET /api/v1/agents/metrics
```

Response

```
{
  "planner": {
    "requests": 0,
    "success_rate": 0
  }
}
```

18. Planner APIs

Planner controls task decomposition.

Generate Task Plan

Endpoint

```
POST /api/v1/agents/planner/generate
```

Request

```
{  
  "task_id": ""  
}
```

Replan

Endpoint

```
POST /api/v1/agents/planner/replan
```

Planner Response

```
{  
  "plan": [],  
  "dependencies": [],  
  "success_criteria": []  
}
```

19. Reasoner APIs

Deep technical reasoning.

Analyze Task

Endpoint

```
POST /api/v1/agents/reasoner/analyze
```

Request

```
{
  "task_id": ""
}
```

Root Cause Analysis

Endpoint

```
POST /api/v1/agents/reasoner/root-cause
```

Generate Patch

Endpoint

```
POST /api/v1/agents/reasoner/generate-patch
```

Impact Analysis

Endpoint

```
POST /api/v1/agents/reasoner/impact-analysis
```

Reasoner Output

```
{
  "findings": [],
  "patches": [],
  "confidence": 0.94
}
```

20. Evidence APIs

Evidence drives reasoning.

Create Evidence

Endpoint

```
POST /api/v1/evidence
```

Get Evidence

Endpoint

```
GET /api/v1/evidence/{evidenceId}
```

Task Evidence

Endpoint

```
GET /api/v1/tasks/{taskId}/evidence
```

Evidence Schema

```
{
  "id": "",
  "source": "",
  "type": "",
  "confidence": 0.0
}
```

21. Findings APIs

Findings represent validated issues.

List Findings

Endpoint

```
GET /api/v1/tasks/{taskId}/findings
```

Get Finding

Endpoint

```
GET /api/v1/findings/{findingId}
```

Update Finding

Endpoint

```
PATCH /api/v1/findings/{findingId}
```

Finding Schema

```
{
  "id": "",
  "summary": "",
  "severity": "",
  "confidence": 0.91
}
```

22. Patch APIs

Patches represent candidate fixes.

Generate Patch

Endpoint

```
POST /api/v1/patches/generate
```

Get Patch

Endpoint

```
GET /api/v1/patches/{patchId}
```

Patch Diff

Endpoint

```
GET /api/v1/patches/{patchId}/diff
```

Approve Patch

Endpoint

```
POST /api/v1/patches/{patchId}/approve
```

Reject Patch

Endpoint

```
POST /api/v1/patches/{patchId}/reject
```

23. Verifier APIs

Independent validation system.

Verify Finding

Endpoint

```
POST /api/v1/agents/verifier/verify-finding
```

Verify Patch

Endpoint

```
POST /api/v1/agents/verifier/verify-patch
```

Regression Review

Endpoint

```
POST /api/v1/agents/verifier/regression-review
```

Final Approval

Endpoint

```
POST /api/v1/agents/verifier/final-approval
```

Verifier Response

```
{
  "approved": true,
  "confidence": 0.96,
  "comments": []
}
```

24. Verification APIs

Create Verification

Endpoint

```
POST /api/v1/verifications
```

Get Verification

Endpoint

```
GET /api/v1/verifications/{verificationId}
```

Verification Status

Endpoint

```
GET /api/v1/verifications/{verificationId}/status
```

Verification Checks

Endpoint

```
GET /api/v1/verifications/{verificationId}/checks
```

25. Antigravity IDE Integration Requirements

These APIs directly power:

Task Workspace

```
/tasks/*
```

Timeline Workspace

```
/tasks/{id}/timeline
```

Memory Workspace

```
/memory/*
```

Findings Workspace

```
/findings/*
```

Patch Review Workspace

```
/patches/*
```

Agent Console

```
/agents/*
```

26. Internal Service Contracts

Planner Output:

```
{
  "task_id": "",
  "plan": []
}
```

Reasoner Output:

```
{
  "findings": [],
```

```
"patches": []  
}
```

Verifier Output:

```
{  
  "approved": true  
}
```

End of API Specification Part 2

Next Section:

Part 3

- Tool APIs
- Browser APIs
- Terminal APIs
- Git APIs
- Repository Intelligence APIs
- Execution APIs
- Report APIs
- Artifact APIs

These APIs will power the actual execution layer inside Antigravity IDE.